

Document Number: P0194R2, ISO/IEC JTC1 SC22 WG21
Date: 2016-10-15
Project: Programming Language C++
Audience: Reflection(SG7) / EWG
Authors: Matúš Chochlík (chochlik@gmail.com)
Axel Naumann (axel@cern.ch)
David Sankel (camior@gmail.com)

Static reflection

Table of Contents

- [Introduction](#)
- [Interplay with other proposals](#)
- [Proposed wording](#)
- [Revision history](#)
- [Acknowledgments](#)

Introduction

This paper is the sixth revision of the proposal to add support for compile-time reflection to standard C++. We propose that the compiler shall generate *metaobjects* — representations of certain program declarations, which can be reasoned-about at compile time. These metaobjects can then be used, through a set of *operations* to obtain various pieces of metadata, like declaration names, lists of scope members, information about specifiers, and so on. The *metaobjects* are implemented as anonymous types conforming to defined concepts, and the operations are implemented as class templates.

This paper is accompanied by another paper — P0385, which discusses the use cases, rationale, design decisions, future evolution of the proposed reflection facility and contains multiple examples of use and replies to frequently asked questions.

Interplay with other proposals

This proposal relies on the Concepts TS, including a resolution to its [Issue 29](#). It also assumes that WG21 will incorporate proper compile-time strings; until then, this proposal provides a placeholder implementation (see [refl.ops.str]) to facilitate reasoning about the interfaces.

Proposed wording

Insert a new element in Table 1, C++ *library headers* of [general.namespaces], named <experimental/reflexpr>.

Insert a new section:

? Compile-time reflection

[refl]

?1 General description

[refl.general]

Compile-time constant metadata describing various aspects of a compiled program are provided indirectly by the means of types generated by the compiler — *Metaobjects*. A *Metaobject* is an anonymous type *reflecting* (representing) a particular declaration or another base-level entity in a C++ program. It can be reasoned-about at compile-time and provides access to *metadata* describing various properties of the reflected entity through a set of templates. The *Metaobject* types themselves are opaque, without any visible internal structure. Values of such a *Metaobject* type can be default-constructed, copy-constructed and copy-assigned.

Metaobjects are generated on demand by the compiler as the result of the invocation of the `reflexpr` operator.

The actual metadata is obtained by instantiating templates constituting the *interface* of the *Metaobjects*. Each template serves a particular purpose and provides a single piece of metadata. These templates are collectively referred to as *Metaobject operations*.

Metaobjects reflecting different kinds of declarations conform to different *Metaobject concepts* and have different interfaces — sets of templates applicable to the *Metaobjects*. These concepts can also be used for *Metaobject* classification.

Metaobject concepts form a generalization-specialization hierarchy, with `meta::Object` being the common generalization for all *Metaobjects*.

?2 *Metaobject* trait

[[refl.trait](#)]

A new *UnaryTypeTrait* — `is_metaobject` is added to section [[meta.unary.cat](#)] to distinguish between *Metaobject* types generated by the compiler and all other types.

```
namespace std {
namespace experimental {

template <typename T> struct is_metaobject;
template <typename T> constexpr bool is_metaobject_v = is_metaobject<T>::value;

} // namespace experimental
} // namespace std
```

[*Example*:

```
is_metaobject_v<int>;           // false
is_metaobject_v<reflexpr(int)>; // true
is_metaobject_v<reflexpr(std)>; // true
using Mstring = reflexpr(string);
is_metaobject_v<Mstring>;       // true
```

— *end example*]

?3 Header `<experimental/reflexpr>` synopsis

[[refl.synopsis](#)]

```
namespace std {
namespace experimental
namespace meta {

// metaobject concepts
template <typename T> concept bool Object;
template <Object T> concept bool ObjectSequence;
template <Object T> concept bool Reversible;
template <Object T> concept bool Named;
template <Object T> concept bool Alias;
template <Object T> concept bool Typed;
template <Object T> concept bool Scope;
template <Object T> concept bool ScopeMember;
template <Object T> concept bool RecordMember;
template <Object T> concept bool EnumMember;
template <Object T> concept bool Namespace;
template <Object T> concept bool GlobalScope;
template <Object T> concept bool Type;
template <Object T> concept bool TagType;
template <Object T> concept bool Record;
template <Object T> concept bool Class;
template <Object T> concept bool Enum;
template <Object T> concept bool Constant;
template <Object T> concept bool Variable;
template <Object T> concept bool Specifier;
template <Object T> concept bool Inheritance;

// metaobject operations
template <Object T1, Object T2> struct reflects_same;
template <Object T> struct get_source_location;
template <ObjectSequence T> struct get_size;
template <ObjectSequence T1, size_t Index> struct get_element;
template <ObjectSequence T1, template <class...> class Tpl> struct unpack_sequence;
template <Named T> struct is_anonymous;
template <Named T> struct get_base_name;
template <Named T> struct get_display_name;
template <Alias T> struct get_aliased;
template <Typed T> struct get_type;
template <ScopeMember T> struct get_scope;
template <RecordMember T> struct is_public;
template <RecordMember T> struct is_protected;
template <RecordMember T> struct is_private;
template <RecordMember T> struct get_access_specifier;
template <Type T> struct get_reflected_type;
template <TagType T> struct get_elaborated_type_specifier;
template <TagType T> struct is_enum;
template <TagType T> struct is_class;
template <TagType T> struct is_struct;
template <TagType T> struct is_union;
template <Record T> struct get_data_members;
```

```

template <Record T> struct get_public_data_members;
template <Record T> struct get_member_types;
template <Record T> struct get_public_member_types;
template <Class T> struct get_base_classes;
template <Class T> struct get_public_base_classes;
template <Enum T> struct is_scoped_enum;
template <Enum T> struct get_enumerators;
template <Constant T> struct get_constant;
template <Variable T> struct is_static;
template <Variable T> struct get_pointer;
template <Inheritance T> struct get_base_class;
template <Inheritance T> struct is_virtual;
template <Inheritance T> struct is_public;
template <Inheritance T> struct is_protected;
template <Inheritance T> struct is_private;
template <Inheritance T> struct get_access_specifier;

//
template <Object T1, Object T2>
    constexpr auto reflects_same_v = reflects_same<T1, T2>::value;
template <ObjectSequence T>
    constexpr auto get_size_v = get_size<T>::value;
template <ObjectSequence T1, size_t Index>
    using get_element_m = typename get_element<T1, Index>::type;
template <ObjectSequence T1, template <class...> class Tpl>
    constexpr auto unpack_sequence_v = unpack_sequence<T1, Tpl>::value;
template <Named T>
    constexpr auto is_anonymous_v = is_anonymous<T>::value;
template <Named T>
    constexpr auto get_base_name_v = get_base_name<T>::value;
template <Named T>
    constexpr auto get_display_name_v = get_display_name<T>::value;
template <Alias T>
    using get_aliased_m = typename get_aliased<T>::type;
template <Typed T>
    using get_type_m = typename get_type<T>::type;
template <ScopeMember T>
    using get_scope_m = typename get_scope<T>::type;
template <RecordMember T>
    constexpr auto is_public_v = is_public<T>::value;
template <RecordMember T>
    constexpr auto is_protected_v = is_protected<T>::value;
template <RecordMember T>
    constexpr auto is_private_v = is_private<T>::value;
template <RecordMember T>
    using get_access_specifier_m = typename get_access_specifier<T>::type;
template <Type T>
    using get_reflected_type_t = typename get_reflected_type<T>::type;
template <TagType T>
    using get_elaborated_type_specifier_m = typename get_elaborated_type_specifier<T>::type;
template <TagType T>
    constexpr auto is_enum_v = is_enum<T>::value;
template <TagType T>
    constexpr auto is_class_v = is_class<T>::value;
template <TagType T>
    constexpr auto is_struct_v = is_struct<T>::value;
template <TagType T>
    constexpr auto is_union_v = is_union<T>::value;
template <Record T>
    using get_data_members_m = typename get_data_members<T>::type;
template <Record T>
    using get_public_data_members_m = typename get_public_data_members<T>::type;
template <Record T>
    using get_member_types_m = typename get_member_types<T>::type;
template <Record T>
    using get_public_member_types_m = typename get_public_member_types<T>::type;
template <Class T>
    using get_base_classes_m = typename get_base_classes<T>::type;
template <Class T>
    using get_public_base_classes_m = typename get_public_base_classes<T>::type;
template <Enum T>
    constexpr auto is_scoped_enum_v = is_scoped_enum<T>::value;
template <Enum T>
    using get_enumerators_m = typename get_enumerators<T>::type;
template <Constant T>
    constexpr auto get_constant_v = get_constant<T>::value;
template <Variable T>
    constexpr auto is_static_v = is_static<T>::value;
template <Variable T>
    const auto get_pointer_v = get_pointer<T>::value;
template <Inheritance T>
    using get_base_class_m = typename get_base_class<T>::type;
template <Inheritance T>
    constexpr auto is_virtual_v = is_virtual<T>::value;
template <Inheritance T>
    constexpr auto is_public_v = is_public<T>::value;
template <Inheritance T>
    constexpr auto is_protected_v = is_protected<T>::value;
template <Inheritance T>
    constexpr auto is_private_v = is_private<T>::value;
template <Inheritance T>
    using get_access_specifier_m = typename get_access_specifier<T>::type;

} // namespace meta
} // namespace experimental
} // namespace std

```

Concept	Requirements	Description
template <typename T> concept bool Object;	is_metaobject_v<T>	meta::Object is a concept modelled by stateless anonymous types generated by the compiler which allow access to metadata reflecting specific program declarations.
template <typename T> concept bool ObjectSequence;	Object<T>	meta::ObjectSequence is an ordered sequence of metaobjects.
template <typename T> concept bool Reversible;	Object<T>	meta::Reversible is a metaobject that can be reverted to the original declaration.
template <typename T> concept bool Named;	Object<T>	meta::Named reflects a named declaration.
template <typename T> concept bool Alias;	Named<T>	meta::Alias reflects a type or namespace alias.
template <typename T> concept bool Typed;	Object<T>	meta::Typed reflects a base-level declaration with a type (for example a variable).
template <typename T> concept bool Scope;	Object<T>	meta::Scope reflects a scope.
template <typename T> concept bool ScopeMember;	Object<T>	meta::ScopeMember reflects a declaration nested in a scope (like the global scope, namespace, class, enum, etc.).
template <typename T> concept bool RecordMember;	ScopeMember<T>	meta::RecordMember reflects a class, struct or union member.
template <typename T> concept bool EnumMember;	ScopeMember<T>	meta::EnumMember reflects a enum member.
template <typename T> concept bool Namespace;	Named<T> Scope<T> ScopeMember<T>	meta::Namespace reflects a namespace.
template <typename T> concept bool GlobalScope;	Namespace<T>	meta::GlobalScope reflects the global scope.
template <typename T> concept bool Type;	Reversible<T> Named<T>	meta::Type reflects a type.
template <typename T> concept bool TagType;	Type<T> Scope<T> ScopeMember<T>	meta::TagType reflects a class or an enum.
template <typename T> concept bool Record;	TagType<T>	meta::Record reflects a class, a scope or a union.
template <typename T> concept bool Class;	Record<T>	meta::Class reflects a class or a struct.
template <typename T> concept bool Enum;	TagType<T>	meta::Enum reflects an enumeration.
template <typename T> concept bool Constant;	Reversible<T> Typed<T>	meta::Constant reflects a compile-time constant value.
template <typename T> concept bool Variable;	Reversible<T> Named<T> Typed<T> ScopeMember<T>	meta::Variable reflects a variable.
template <typename T> concept bool Specifier;	Named<T>	meta::Specifier reflects a specifier (const, virtual, static, noexcept, public, protected, private, etc.).
template <typename T> concept bool Inheritance;	Object<T>	meta::Inheritance reflects class inheritance.

[*Note: Metaobjects* reflecting anonymous types, anonymous namespaces and the global scope also satisfy the meta::Named concept. The fact that they reflect an anonymous declaration is indicated by the meta::is_anonymous operation. In such case the meta::get_base_name and the meta::get_display_name operations return an empty (null terminated) string. — *end note*]

[*Example:*

```
static_assert(meta::Named<reflexpr(string)>, "");
auto l = [](void) { };

static_assert(meta::Named<reflexpr()>, "");
static_assert(meta::Named<reflexprdecltype(l)>, "");

static_assert(!meta::is_anonymous_v<reflexpr(int)>, "");
auto l = [](void) { };
static_assert(meta::is_anonymous_v<reflexpr::>, "");
```

```
static_assert(meta::is_anonymous_v<reflexpr(decltype(l))>, "");
static_assert(meta::get_base_name_v<reflexpr(int)>[0] != '\0', "");
auto l = [](void) { };
static_assert(meta::get_base_name_v<reflexpr(:)>[0] == '\0', "");
static_assert(meta::get_base_name_v<reflexpr(decltype(l))>[0] == '\0', "");
```

— end example]

Metaobjects reflecting fundamental types (except for `std::nullptr_t`), pointers (including pointers-to-members), references, arrays and function types do not conform to the *meta::ScopeMember* concept, all other types do.

[Example:

```
struct S { };
namespace foo { class C { }; }

static_assert(!meta::ScopeMember(reflexpr(int)), "");
static_assert(!meta::ScopeMember(reflexpr(int*)), "");
static_assert(!meta::ScopeMember(reflexpr(S[10])), "");
static_assert(!meta::ScopeMember(reflexpr(foo::C&)), "");
static_assert(!meta::ScopeMember(reflexpr(int*)(char, bool)), "");

static_assert( meta::ScopeMember(reflexpr(S)), "");
static_assert( meta::ScopeMember(reflexpr(foo::C)), "");
static_assert( meta::ScopeMember(reflexpr(std::size_t)), "");
```

— end example]

?5 Metaobject operations

[refl.ops]

A *MetaobjectOperation* extracts information from *Metaobjects*. It is a class template taking one or more arguments, at least one of which models the *Metaobject* concept. The result of a *MetaobjectOperation* can be either a compile-time constant value, a `const` pointer, or a type.

?5.1 Boolean constant result

[refl.ops.bool]

All *MetaobjectOperation* templates returning a boolean value derive publicly and unambiguously from `true_type` OR `false_type`.

[Example:

```
struct MetaobjectOperation
: integral_constant<bool, implementation-defined>
{ };
```

— end example]

?5.2 Integral or enum constant result

[refl.ops.int]

All *MetaobjectOperation* templates returning an integer or `enum` value derive publicly and unambiguously from a specialization of `integral_constant`. The first argument's integer or `enum` type is part of the specification of the operation.

[Example:

```
struct MetaobjectOperation
: integral_constant<integral-or-enum-type, implementation-defined>
{ };
```

— end example]

?5.3 Pointer result

[refl.ops.ptr]

The *MetaobjectOperation* templates returning a pointer or a class data member pointer derive publicly and unambiguously from a specialization of `integral_constant` with value as if taken by the `&` operator. The first argument's pointer or class data member pointer type is part of the specification of the operation.

[Example:

```
struct MetaobjectOperation
: integral_constant<pointer-or-data-member-pointer-type, implementation-defined>
{ };
```

— end example]

?5.4 String constant result

[refl.ops.str]

All *MetaobjectOperation* templates returning a character string instantiate into a type *equivalent* to the following:

```
struct MetaobjectOperation
{
    typedef const char value_type[N+1];
    static constexpr value_type value = implementation-defined;
};
```

where *N* is the length of the returned string and the last element in the *value* character array is the `'\0'` character. The member names *value_type* and *value* are publicly and unambiguously available in such *MetaobjectOperation*.

?5.5 Type result

[refl.ops.type]

The *MetaobjectOperation* templates yielding a non-*Metaobject* type instantiates into a type *equivalent* to the following:

```
struct MetaobjectOperation
{
    using type = unspecified;
};
```

The *MetaobjectOperation* templates yielding another *Metaobject* type instantiate into a type *equivalent* to the following:

```
struct MetaobjectOperation
{
    using type = Object;
};
```

The member name *type* is publicly and unambiguously available in such *MetaobjectOperation*.

?5.6 Source location result

[refl.ops.srcloc]

The *MetaobjectOperation* templates yielding *source_location* compile-time values will be equivalent to the following:

```
struct MetaobjectOperation
: source_location /* initialized in an implementation-defined way */
{ };
```

?5.7 Operation description

[refl.ops.desc]

Template	Description	Result
template <Object T1, Object T2> struct reflects_same;	indicates whether two metaobjects reflect the same base-level declaration ('entity').	a boolean constant
template <Object T> struct get_source_location;	returns the source location info of the declaration of a base-level program declaration reflected by a <i>meta::Object</i> .	a <i>std::source_location</i> constant
template <ObjectSequence T> struct get_size;	returns the number of elements in the sequence.	a <i>size_t</i> integral constant
template <ObjectSequence T1, size_t Index> struct get_element;	returns the <i>i</i> -th <i>meta::Object</i> in a <i>meta::ObjectSequence</i> .	<i>meta::Object</i>
template <ObjectSequence T1, template <class...> class Tp1> struct unpack_sequence;	unpacks the <i>meta::ObjectSequence</i>	

	into a template.	a type
<pre>template <Named T> struct is_anonymous;</pre>	indicates whether the a named declaration reflected by a <code>meta::Named</code> is anonymous.	a boolean constant
<pre>template <Named T> struct get_base_name;</pre>	returns the basic name of the a named declaration reflected by a <code>meta::Named</code> .	a string constant
<pre>template <Named T> struct get_display_name;</pre>	returns the display name of the a named declaration reflected by a <code>meta::Named</code> .	a string constant
<pre>template <Alias T> struct get_aliased;</pre>	returns the <code>meta::Named</code> reflecting the original declaration of a type or namespace alias reflected by a <code>meta::Alias</code> .	<code>meta::Named</code>
<pre>template <Typed T> struct get_type;</pre>	returns the <code>meta::Type</code> reflecting the type of a base-level declaration with a type (for example a variable) reflected by a <code>meta::Typed</code> .	<code>meta::Type</code>
<pre>template <ScopeMember T> struct get_scope;</pre>	returns the <code>meta::Scope</code> reflecting the scope of a declaration nested in a scope (like the global scope, namespace, class, enum, etc.) reflected by a <code>meta::ScopeMember</code> .	<code>meta::Scope</code>
<pre>template <RecordMember T> struct is_public;</pre>	indicates whether the a class, struct or union member reflected by a <code>meta::RecordMember</code> was declared with public access.	a boolean constant
<pre>template <RecordMember T> struct is_protected;</pre>	indicates whether the a class, struct or union member reflected by a <code>meta::RecordMember</code> was declared with protected access.	a boolean constant
<pre>template <RecordMember T> struct is_private;</pre>	indicates whether the a class, struct or union member reflected by	a boolean constant

	a <code>meta::RecordMember</code> was declared with private access.	
<pre>template <RecordMember T> struct get_access_specifier;</pre>	returns the <code>meta::Specifier</code> reflecting the access specifier of a class, struct or union member reflected by a <code>meta::RecordMember</code> .	<code>meta::Specifier</code>
<pre>template <Type T> struct get_reflected_type;</pre>	returns the the base-level type reflected by a <code>meta::Type</code> .	the original type reflected by the argument
<pre>template <TagType T> struct get_elaborated_type_specifier;</pre>	returns a <code>meta::Specifier</code> reflecting the elaborated type specifier (class,struct,union,...) of a class or an enum reflected by a <code>meta::TagType</code> .	<code>meta::Specifier</code>
<pre>template <TagType T> struct is_enum;</pre>	indicates whether the a class or an enum reflected by a <code>meta::TagType</code> is an enum.	a boolean constant
<pre>template <TagType T> struct is_class;</pre>	indicates whether the a class or an enum reflected by a <code>meta::TagType</code> is a class.	a boolean constant
<pre>template <TagType T> struct is_struct;</pre>	indicates whether the a class or an enum reflected by a <code>meta::TagType</code> is a struct.	a boolean constant
<pre>template <TagType T> struct is_union;</pre>	indicates whether the a class or an enum reflected by a <code>meta::TagType</code> is a union.	a boolean constant
<pre>template <Record T> struct get_data_members;</pre>	returns a <code>meta::ObjectSequence</code> of <code>meta::Variable(s)</code> reflecting all (including private and protected) data members of a class, a scope or a union reflected by a <code>meta::Record</code> .	<code>meta::ObjectSequence</code> of <code>meta::Variable(s)</code>
<pre>template <Record T> struct get_public_data_members;</pre>	returns a <code>meta::ObjectSequence</code> of <code>meta::Variable(s)</code> reflecting the public data members of a class, a scope or a union reflected by a <code>meta::Record</code> .	<code>meta::ObjectSequence</code> of <code>meta::Variable(s)</code>
	returns a	

<pre>template <Record T> struct get_member_types;</pre>	<code>meta::ObjectSequence</code> of <code>meta::Type(s)</code> reflecting all (including private and protected) member types of a class, a scope or a union reflected by a <code>meta::Record</code>.	<code>meta::ObjectSequence</code> of <code>meta::Type(s)</code>
<pre>template <Record T> struct get_public_member_types;</pre>	returns a <code>meta::ObjectSequence</code> of <code>meta::Type(s)</code> reflecting the public member types of a class, a scope or a union reflected by a <code>meta::Record</code>.	<code>meta::ObjectSequence</code> of <code>meta::Type(s)</code>
<pre>template <Class T> struct get_base_classes;</pre>	returns a <code>meta::ObjectSequence</code> of <code>meta::Inheritance(s)</code> reflecting the inheritance of all base classes (including the private and protected ones) of a class or a struct reflected by a <code>meta::Class</code>.	<code>meta::ObjectSequence</code> of <code>meta::Inheritance(s)</code>
<pre>template <Class T> struct get_public_base_classes;</pre>	returns a <code>meta::ObjectSequence</code> of <code>meta::Inheritance(s)</code> reflecting the inheritance of public base classes of a class or a struct reflected by a <code>meta::Class</code>.	<code>meta::ObjectSequence</code> of <code>meta::Inheritance(s)</code>
<pre>template <Enum T> struct is_scoped_enum;</pre>	returns true if the an enumeration reflected by a <code>meta::Enum</code> is scoped.	a boolean constant
<pre>template <Enum T> struct get_enumerators;</pre>	returns a <code>meta::ObjectSequence</code> of <code>meta::Constant(s)</code> reflecting all enumerators of an enumeration reflected by a <code>meta::Enum</code>.	<code>meta::ObjectSequence</code> of <code>meta::Constant(s)</code>
<pre>template <Constant T> struct get_constant;</pre>	returns the value of a compile-time constant value reflected by a <code>meta::Constant</code>.	a constant value of an unspecified, integer or enum type
<pre>template <Variable T> struct is_static;</pre>	returns whether the a variable reflected by a <code>meta::Variable</code> was declared with	a boolean constant

	the static specifier.	
<pre>template <Variable T> struct get_pointer;</pre>	returns a pointer to the a variable reflected by a <code>meta::Variable</code> . If the variable is a class member then the pointer is a class data member pointer, otherwise it is a plain pointer.	a regular or class data member pointer
<pre>template <Inheritance T> struct get_base_class;</pre>	returns the <code>meta::Inheritance</code> reflecting the base class inheritance in a inheritance.	<code>meta::Class</code>
<pre>template <Inheritance T> struct is_virtual;</pre>	returns whether the class inheritance reflected by a <code>meta::Inheritance</code> was declared as virtual.	a boolean constant
<pre>template <Inheritance T> struct is_public;</pre>	indicates whether the class inheritance reflected by a <code>meta::Inheritance</code> was declared as public.	a boolean constant
<pre>template <Inheritance T> struct is_protected;</pre>	indicates whether the class inheritance reflected by a <code>meta::Inheritance</code> was declared as protected.	a boolean constant
<pre>template <Inheritance T> struct is_private;</pre>	indicates whether the class inheritance reflected by a <code>meta::Inheritance</code> was declared as private.	a boolean constant
<pre>template <Inheritance T> struct get_access_specifier;</pre>	returns the <code>meta::Specifier</code> reflecting the access specifier of class inheritance reflected by a <code>meta::Inheritance</code> .	<code>meta::Specifier</code>

?5.7.1 Operation `get_base_name`

[**refl.ops.base_name**]

For `meta::Types` reflecting fundamental types, the `get_base_name` operation returns the longest type specifier name, i.e. including long, signed OR unsigned modifiers, if they describe the identical type.

[Example:

```
meta::get_base_name_v<reflexpr(unsigned)>      // "unsigned int"
meta::get_base_name_v<reflexpr(short)>         // "signed short int"
meta::get_base_name_v<reflexpr(long long)>      // "signed long long int"
meta::get_base_name_v<reflexpr(long double)>    // "long double"
meta::get_base_name_v<reflexpr(signed char)>     // "signed char"
meta::get_base_name_v<reflexpr(unsigned char)>  // "unsigned char"
meta::get_base_name_v<reflexpr(char)>           // "char"
```

— end example]

The `get_base_name` invoked on a `meta::Alias` returns the alias name, not the name of the aliased declaration.

[Example:

```
using foo = int;
using bar = foo;
namespace baz = std;

typedef foo(*pfoo)(double, float);

meta::get_base_name_v<reflexpr(foo)>      // "foo"
meta::get_base_name_v<reflexpr(bar)>      // "bar"
meta::get_base_name_v<reflexpr(baz)>      // "baz"
meta::get_base_name_v<reflexpr(pfoo)>     // "pfoo"
```

— end example]

The `get_base_name` operation returns the type, namespace, etc. name without any qualifiers, asterisks, ampersands, angle or square brackets, etc.

[Example:

```
using meta::get_base_name_v;

get_base_name_v<reflexpr(std::vector<int>)>      // "vector"
get_base_name_v<reflexpr(volatile std::size_t* [10])> // "unsigned long int"
get_base_name_v<reflexpr(std::set<int>* (std::vector<double>&))> // ""
get_base_name_v<reflexpr(std::chrono)>          // "chrono"
get_base_name_v<reflexpr(std::launch)>          // "launch"
get_base_name_v<reflexpr(static)>               // "static"
get_base_name_v<reflexpr(class)>                // "class"
```

— end example]

?5.7.2 Operation `get_display_name`

[[refl.ops.display_name](#)]

The `meta::get_display_name` operation returns an implementation-defined string containing the name of the base-level entity reflected by a `meta::Named` which is as close as possible to how the name is spelled in the source code. For types in particular the display name should be fully qualified and if possible it should preserve type or template alias names.

[Example:

```
meta::get_display_name_v<reflexpr(int)>          // "int"
meta::get_display_name_v<reflexpr(const std::string&)> // "const std::string&"
meta::get_display_name_v<reflexpr(std::size_t * [10])> // "std::size * [10]"
meta::get_display_name_v<reflexpr(decltype("foo"))> // "char const [4]"
```

— end example]

?5.7.3 Operation `get_aliased`

[[refl.ops.aliased](#)]

The `meta::get_aliased` operation invoked on a `meta::Alias` always returns a (non `meta::Alias`) `meta::Type` or `meta::Namespace` reflecting the entity which has a unique identity at the base-level, which is directly or indirectly referenced by the original alias.

[Example:

```
using A = int;
using B = const A;
using C = B*;

template <typename T>
struct S
{
    using X = T&;
};

static_assert( meta::Type<reflexpr(A)>, "");
static_assert( meta::Alias<reflexpr(A)>, "");
static_assert( meta::Type<meta::get_aliased_m<reflexpr(A)>>, "");
static_assert( meta::Alias<meta::get_aliased_m<reflexpr(A)>>, "");

static_assert( meta::Type<reflexpr(B)>, "");
static_assert( meta::Alias<reflexpr(B)>, "");
static_assert( meta::Type<meta::get_aliased_m<reflexpr(B)>>, "");
static_assert( !meta::Alias<meta::get_aliased_m<reflexpr(B)>>, "");

static_assert( meta::Type<reflexpr(C)>, "");
static_assert( meta::Alias<reflexpr(C)>, "");
static_assert( meta::Type<meta::get_aliased_m<reflexpr(C)>>, "");
static_assert( !meta::Alias<meta::get_aliased_m<reflexpr(C)>>, "");
```

```

static_assert(meta::reflects_same_v<
    meta::get_aliased_m<reflexpr(A)>,
    reflexpr(int)
>, "");

static_assert(meta::reflects_same_v<
    meta::get_aliased_m<reflexpr(B)>,
    reflexpr(const int)
>, "");

static_assert(meta::reflects_same_v<
    meta::get_aliased_m<reflexpr(C)>,
    reflexpr(const int *)
>, "");

static_assert(meta::reflects_same_v<
    meta::get_aliased_m<reflexpr(S<A>::X)>,
    reflexpr(int &)
>, "");

static_assert(meta::reflects_same_v<
    meta::get_aliased_m<reflexpr(S<B>::X)>,
    reflexpr(const int &)
>, "");

static_assert(meta::reflects_same_v<
    meta::get_aliased_m<reflexpr(S<C>::X)>,
    reflexpr(const int * &)
>, "");

```

— *end example*

Template type parameters are also considered to be a special kind of type aliases:

[*Example:*

```

template <typename A, typename B>
void foo(A a, B b)
{
    using meta::get_base_name_v;
    using meta::get_aliased_m;

    using mA = reflexpr(A);
    using mB = reflexpr(B);

    cout << "1:" << get_base_name_v<mA> << endl;
    cout << "2:" << get_base_name_v<get_aliased_m<mA>> << endl;

    cout << "3:" << get_base_name_v<mB> << endl;
    cout << "4:" << get_base_name_v<get_aliased_m<mB>> << endl;
}

using rank_t = int;
rank_t a = 123;
float b = 45.67;

foo(a, b);

using mR = reflexpr(rank_t);
cout << "5:" << get_base_name_v<mR> << endl;
cout << "6:" << get_base_name_v<get_aliased_m<mR>> << endl;

```

produces the following output:

```

1:A
2:int
3:B
4:float
5:rank_t;
6:int;

```

— *end example*

?5.7.4 Sequence operations

[[refl.ops.seq](#)]

Metaobjects in a `meta::ObjectSequence` are ordered according to the order of each *Metaobject*'s first declaration in the translation unit.

[*Example:*

```

struct foo
{
    typedef int t1;
    typedef float t2;

    t1 a, b, c;
    t2 d;
};

using meta_foo = reflexpr(foo);

```

```

using meta::get_base_name_v;
using meta::get_element_m;
using meta::get_data_members_m;
using meta::get_member_types_m;

get_base_name_v<get_element_m<get_member_types_m<meta_foo>, 0>> // "t1"
get_base_name_v<get_element_m<get_member_types_m<meta_foo>, 1>> // "t2"

get_base_name_v<get_element_m<get_data_members_m<meta_foo>, 0>> // "a"
get_base_name_v<get_element_m<get_data_members_m<meta_foo>, 1>> // "b"
get_base_name_v<get_element_m<get_data_members_m<meta_foo>, 2>> // "c"
get_base_name_v<get_element_m<get_data_members_m<meta_foo>, 3>> // "d"

```

— end example]

[Note: The name of `get_element` may be shortened to `get` or unified with an equivalent operation on type-lists if they become part of the standard in the future. — end note]

The `meta::ObjectSequence` returned by `meta::get_data_members`, `meta::get_public_data_members`, `meta::get_member_types` and `meta::get_public_member_types` *do not* include *Metaobjects* reflecting inherited class members.

[Example:

```

struct foo
{
    typedef int attr_t;
    attr_t a, b, c, d;
};

struct bar : foo
{
    float e, f;
};

using meta_foo = reflexpr(foo);
using meta_bar = reflexpr(bar);

using meta::get_size_v;
using meta::get_data_members_m;
using meta::get_member_types_m;

get_size_v<get_data_members_m<meta_foo>> // 4
get_size_v<get_data_members_m<meta_bar>> // 2

get_size_v<get_member_types_m<meta_foo>> // 1
get_size_v<get_member_types_m<meta_bar>> // 0

```

— end example]

The `meta::ObjectSequence` returned by `meta::get_data_members` and `meta::get_member_types` contain *Metaobjects* reflecting all class members including those which are not publicly accessible otherwise. The `meta::ObjectSequence` returned by `meta::get_public_data_members` and `meta::get_public_member_types` contain only the *Metaobjects* reflecting publicly accessible class members.

[Example:

```

struct foo
{
private:
    typedef int _attr_t;
    _attr_t _a, _b, _c;
public:
    typedef float value_type;
    typedef size_t size_type;
    value_type d;
};

using meta_foo = reflexpr(foo);

using meta::get_size_v;
using meta::get_base_name_v;
using meta::get_element_m;
using meta::get_data_members_m;
using meta::get_public_data_members_m;
using meta::get_member_types_m;
using meta::get_public_member_types_m;

get_size_v<get_data_members_m<meta_foo>> // 4
get_size_v<get_public_data_members_m<meta_foo>> // 1

get_size_v<get_member_types_m<meta_foo>> // 3
get_size_v<get_public_member_types_m<meta_foo>> // 2

get_base_name_v<get_element_m<get_data_members_m<meta_foo>, 0>> // "_a"
get_base_name_v<get_element_m<get_public_data_members_m<meta_foo>, 0>> // "d"

```

— end example]

The `unpack_sequence` operation is applied to models of `meta::ObjectSequence`. It is *equivalent* to:

```

namespace std {
namespace experimental {
namespace meta {

template <ObjectSequence Seq, template <typename ...> class Tpl>
struct unpack_sequence
{
    typedef Tpl<
        get_element_m<Seq, 0>,
        get_element_m<Seq, 1>,
        ...,
        get_element_m<Seq, get_size_v<Seq>-1>,
    > type;
};

template <ObjectSequence Seq, template <typename ...> class Tpl>
using unpack_sequence_t = typename unpack_sequence<Seq, Tpl>::type;

} // namespace meta
} // namespace experimental
} // namespace std

```

[Example:

```

struct foo
{
    int a;
    bool b;
    char c;
    double d;
};

template <meta::DataMember ... MDM>
using helper = tuple<meta::get_reflected_type_t<MDM>...>;

using X = meta::unpack_sequence_t<
    meta::get_data_members_m<reflexpr(foo)>,
    helper
>;

is_same_v<X, tuple<int, bool, char, double>> // true

```

— end example]

?5.7.5 Specifier operations

[refl.ops.spec]

[Note: There is a dual interface for retrieving specifier-related metadata. One is using the `meta::Specifier` concept and its operations which is more generic, and another using the boolean operations like `meta::is_static`, `meta::is_public`, etc. which may be more convenient in simpler use-cases. — end note]

[Example:

```

struct S
{
private:
    int i;
public:
    float f;
};

using meta_S = reflexpr(S);
using meta_S_i = meta::get_element_m<meta::get_data_members_m<meta_S>, 0>;
using meta_S_f = meta::get_element_m<meta::get_data_members_m<meta_S>, 1>;

using meta::reflects_same_v;
using meta::get_access_specifier_m;

reflects_same_v<reflexpr(private), get_access_specifier_m<meta_S_i>> // true
reflects_same_v<reflexpr(public), get_access_specifier_m<meta_S_i>> // false
reflects_same_v<reflexpr(public), get_access_specifier_m<meta_S_f>> // true

meta::is_private_v<meta_S_i> // true
meta::is_public_v<meta_S_i> // false
meta::is_public_v<meta_S_f> // true

```

— end example]

?6 The reflection operator

[refl.reflexpr]

The result of the invocation of the `reflexpr` is a type satisfying `meta::Object` and other `meta` concepts, depending on the operand. The returned type is implementation-defined. Meta-operations on it describe the operand.

The `reflexpr` operator takes an *id-expression* (5.1), or *access-specifier* (10) as operand, or nothing. The *id-expression* must be uniquely resolved through regular lookup, as if used outside of the `reflexpr` operator. Otherwise, the expression is ill-formed.

- If no operand is passed to `reflexpr` or if the operand is `::`, then the result will be satisfying `meta::GlobalScope`.
- Otherwise, if the operand is a *union-name*, the result will be satisfying `meta::Record`.
- Otherwise, if the operand is a *class-name*, the result will be satisfying `meta::Class`.
- Otherwise, if the operand is a *enum-name*, the result will be satisfying `meta::Enum`.
- Otherwise, if the operand is a *type-name*, the result will be satisfying `meta::Type`.
- Otherwise, if the operand is a *typedef-name*, the result will be satisfying `meta::Type` and `meta::Alias`.
- Otherwise, if the operand is a *namespace-name*, the result will be satisfying `meta::Namespace`.
- Otherwise, if the operand is the name of a *namespace-alias-declaration*, the result will be satisfying `meta::Namespace` and `meta::Alias`.
- Otherwise, if the operand is a *class-data-member-name*, the result will be satisfying `meta::Variable`.
- Otherwise, if the operand is a *variable-name*, the result will be satisfying `meta::Variable`.
- Otherwise, if the operand is a *enumerator-name*, the result will be satisfying `meta::Constant`.
- Otherwise, if the operand is any of `static`, `virtual`, `enum`, `class`, `struct`, `union`, `public`, `protected`, `private`, the result will be satisfying `meta::Specifier`.

If the scope of the declaration of the *id-expression* is a class, a struct or a union then the result of `reflexpr` is also satisfying `meta::RecordMember`.

If the scope of the declaration of the *id-expression* is an enum then the result of `reflexpr` is also satisfying `meta::EnumMember`.

The invocation of `reflexpr` on all other kinds of *id-expressions* is ill-formed. The proposal will be extended in the future revisions to allow additional operands.

[Example:

```
reflexpr() // reflects the global namespace
reflexpr::) // reflects the global namespace
reflexpr(std) // reflects the namespace
reflexpr(int) // reflects the fundamental type
reflexpr(std::size_t) // reflects the type alias
reflexpr(std::launch) // reflects the enum type
reflexpr(std::vector<int>) // reflects the class
reflexpr(std::pair<int, int>::first_type) // reflects the member typedef
reflexpr(std::pair<int, int>::first) // reflects the data member
reflexpr(std::launch::async) // reflects the enumerator

reflexpr(1) // ill-formed
reflexpr(std::sin) // ill-formed
reflexpr(std::vector) // ill-formed
reflexpr(is_same_v<void, void>) // ill-formed
```

— end example]

[Note: The expressions `reflexpr()` and `reflexpr::)` both reflect the global scope. It is possible that in the future only one of them will be chosen for this purpose. — end note]

?6.1 Redclarations

[refl.reflexpr.redecl]

The meta data queried by `reflexpr` depends on the point of invocation of the operator; only declarations appearing in the translation unit before the invocation of the `reflexpr` operator will be visible. Subsequent invocations are independent of prior invocations, as if all compiler generated types were unique to each `reflexpr` invocation.

[Example:

```
using meta::get_size_v;
using meta::get_data_members_m;

struct foo;
using meta_foo_fwd1 = reflexpr(foo);
constexpr size_t n1 = get_size_v<get_data_members_m<meta_foo_fwd1>>; // 0

struct foo;
using meta_foo_fwd2 = reflexpr(foo);
constexpr size_t n2 = get_size_v<get_data_members_m<meta_foo_fwd2>>; // 0

using meta::reflects_same_v;

constexpr bool b1 = is_same_v<meta_foo_fwd1, meta_foo_fwd2>; // unspecified
constexpr bool b2 = reflects_same_v<meta_foo_fwd1, meta_foo_fwd2>; // true

struct foo { int a,b,c,d; };
using meta_foo = reflexpr(foo);
constexpr size_t n3 = get_size_v<get_data_members_m<meta_foo>>; // 4

constexpr bool b3 = is_same_v<meta_foo_fwd1, meta_foo>; // false
constexpr bool b4 = reflects_same_v<meta_foo_fwd1, meta_foo>; // true
```

— *end example*]

[*Note:* In order to distinguish the reflections of incomplete forward declarations of types a new concept like `meta::IncompleteType` Or `meta::ForwardDeclared` or a new operation like `meta::is_forward_declared` will be added in a future revision. — *end note*]

Revision history

Revision 1 ([N3996](#))

Describes the method of static reflection by the means of compiler-generated anonymous types. Introduces the first version of the metaobject concepts and some possibilities of their implementation. Also includes discussion about the motivation and the design rationale for the proposal.

Revision 2 ([N4111](#))

Refines the metaobject concepts and introduces a concrete implementation of their interface by the means of templates similar to the standard type traits. Describes some additions to the standard library (mostly meta-programming utilities), which simplify the use of the metaobjects. Answers some questions from the discussion about N3996 and expands the design rationale.

Revision 3 ([N4451](#))

Incorporates the feedback from the discussion about N4111 at the Urbana meeting, most notably reduces the set of metaobject concepts and refines their definitions, removes some of the additions to the standard library added in the previous revisions. Adds context-dependent reflection.

Revision 4 ([P0194R0](#))

Further refines the concepts from N4111; prefixes the names of the metaobject operations with `get_`, adds new operations, replaces the metaobject category tags with new metaobject traits. Introduces a nested namespace `std::meta` which contains most of the reflection-related additions to the standard library. Rephrases definition of meta objects using Concepts Lite. Specifies the reflection operator name — `reflexpr`. Introduces an experimental implementation of the reflection operator in clang. Drops the context-dependent reflection from N4111 (will be re-introduced later).

Revision 5 ([P0194R1](#))

Dropped all metaobject traits except `is_metaobject`. All metaobject classification is now done by using the concepts. We assume that the following syntax of the Concepts specification is implemented:

```
template <T>
constexpr bool Concept = unspecified;
```

and also assuming that the Concepts TS Issue 29 is resolved and that `Concept<T>` is a generally usable boolean expression, so that the following is valid code:

```
static_assert(Concept<T>, " ... ");
using U = std::conditional_t<Concept<T>, T1, T2>;
if(Concept<T>) { /* ... */ }
```

The `meta::Scoped` concept has been renamed to `meta::ScopeMember`. The `meta::Constant` and `meta::Specifier` concepts, and several new operations have been added.

The aliases for the operation templates returning metaobjects had previously the `_t` suffix; this has been changed to the `_m` suffix. For example:

```
get_type_t -> get_type_m
get_scope_t -> get_scope_m
get_aliased_t -> get_aliased_m
get_data_members_t -> get_data_members_m
```

Revision 6 ([P0194R2](#))

The following concepts from P0194R1 were dropped in order to simplify the proposal: `meta::Linkable`, `meta::Enumerator`, `meta::DataMember`, `meta::MemberType`, `meta::EnumClass`, `meta::TypeAlias` and `meta::NamespaceAlias`.

The following concepts were added to the proposal: `meta::TagType`, `meta::Record`, `meta::EnumMember`.

Unlike in the previous proposal, metaobjects reflecting anonymous entities - the global scope, anonymous namespaces and classes, etc. *do* conform to the `meta::Named` concept and implement the name-returning operations.

Unlike in the previous proposal, metaobjects reflecting the global scope *do* conform to the `meta::ScopeMember` concept and the `meta::get_scope` operation. For arguments reflecting the global scope returns a metaobject reflecting the global scope (i.e. the global scope is its own scope).

Metaobjects reflecting built-in types and types like pointers, references, arrays, etc. now don't have a scope (i.e. they do not conform to the `meta::ScopeMember` concept).

We have added a different mechanism for distinguishing between non-scoped and scoped enums - the `meta::is_scoped_enum` operation. Unlike in the previous proposal `meta::Enum` reflecting a non-scoped enum is a `meta::Scope`.

We now allow the default construction and copy construction of values of metaobject types.

Direct reflection of class data members, member types and type aliases, enumerators and global scope/namespace-level variables has been added. For example:

```
enum class E { a, b, c, d};
using ME_a = reflexpr(E::a); // valid

struct S { using T = int; T a; };
using MS_T = reflexpr(S::T); // valid
using MS_a = reflexpr(S::a); // valid
```

The typedef (type-alias) reflection has been simplified based on the feedback from Oulu. Previously we required reflection to be aware about all aliases in a "*chain*" even in the context of templates. For example:

```
using A = int;
using B = A;
using C = B;

template <typename T>
struct S
{
    using X = T;
};

using Mint = reflexpr(int);
using MA = reflexpr(A);
using MB = reflexpr(B);
using MC = reflexpr(C);

using MSintX = reflexpr(S<int>::X);
using MSAX = reflexpr(S<A>::X);
using MSBX = reflexpr(S<B>::X);
using MSCX = reflexpr(S<C>::X);
```

In the previous proposals we required the following (i.e. that reflection should be aware of the whole "*chain*" of aliases - `S<C>::X -> C -> B -> A -> int`):

```
static_assert(meta::reflects_same_v<meta::get_aliased_m<MSCX>, MC>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MSBX>, MB>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MSAX>, MA>, "");

static_assert(meta::reflects_same_v<meta::get_aliased_m<meta::get_aliased_m<MSCX>>, MB>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<meta::get_aliased_m<meta::get_aliased_m<MSCX>>>, MA>, "");

static_assert(meta::reflects_same_v<meta::get_aliased_m<MC>, MB>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MB>, MA>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MA>, Mint>, "");
```

Now we require only that the type alias knows the "*canonical*" type which it refers to - the type which has a unique identity at the base level (i.e. `S<C>::X -> int`, `C -> int`, `B -> int`, `A -> int`):

```
static_assert(meta::Alias<MSCX>, "");
static_assert(meta::Alias<MSBX>, "");
static_assert(meta::Alias<MSAX>, "");
```

```
static_assert(meta::Alias<MC>, "");
static_assert(meta::Alias<MB>, "");
static_assert(meta::Alias<MA>, "");

static_assert(!meta::Alias<Mint>, "");

static_assert(meta::reflects_same_v<meta::get_aliased_m<MSCX>, Mint>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MSBX>, Mint>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MSAX>, Mint>, "");

static_assert(meta::reflects_same_v<meta::get_aliased_m<MC>, Mint>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MB>, Mint>, "");
static_assert(meta::reflects_same_v<meta::get_aliased_m<MA>, Mint>, "");
```

The mechanism for enumerating public-only vs. all (including non-public ones) class members has been changed. Now the *"basic"* operations like `meta::get_data_members`, `meta::get_member_types`, etc. return all members, and the `meta::get_public_data_members`, `meta::get_public_member_types`, return only the public class members.

Acknowledgments

Thanks to Ricardo Fabiano de Andrade, Roland Bock and Klaim - Joël Lamotte who provided valuable feedback, criticism and suggestions.

References

- [P0194R1] Chochlík M., Naumann A. — *P0194R1 - Static reflection (revision 5)*
- [P0194R0] Chochlík M., Naumann A. — *P0194R0 - Static reflection (revision 4)*
- [N4451] Chochlík M. — *N4451 - Static reflection (revision 3)*
- [N4111] Chochlík M. — *N4111 - Static reflection (revision 2)*
- [N3996] Chochlík M. — *N3996 - Static reflection*
- [P0385R0] Chochlík M., Naumann A. — *P0385R0 - Static reflection - Rationale, design and evolution*
- [P0385R1] Chochlík M., Naumann A. — *P0385R1 - Static reflection - Rationale, design and evolution*